Signature Based Machine Learning Techniques for Realized Volatility Forecasting

Ryan Gu
University of California, Berkeley
ryangu1232@berkeley.edu

Abstract

Our paper introduces a signature based approach for forecasting volatility. Specifically, we decompose the path of an underlying price through a lead-lag transformation, compute the signature of this transformed path, and apply existing machine learning techniques on this signature feature set to predict volatility. This paper will give a brief primer on signature concepts required for this paper, existing volatility forecasting models, a description of the proposed model, and comparisons of the proposed to existing models with numerical experiments.

1 Introduction

Importance of Forecasting Volatility Volatility is one of the most important quantities in financial markets. It is the primary input in most derivatives pricing models, a core component of portfolio risk management, and a key indicator for market stress. As a result, accurate volatility forecasts allow institutions to price options more precisely, construct better-hedged portfolios, and anticipate periods of elevated market risk before they happen.

Despite this importance, volatility forecasting is a difficult problem for many reasons. First, volatility is a latent process meaning it can only be inferred from realized price movements. Second, volatility is complex. It clusters in time, mean-reverts over longer horizons, responds asymmetrically to positive and negative returns, and has long memory properties that persist across timescales. Such complexities make it hard to capture all the effects within a single model. Third, volatility behaves very differently across market regimes. A model trained on calmer periods will not be effective during periods of high volatility, making out-of-sample generalization a challenge. These challenges motivate the ongoing search for forecasting methods that can capture both complex dynamics generalize across regimes.

Motivation. Existing literature suggests that volatility is best modeled by stochastic differential equations (SDEs) such as the Heston, Volterra, SABR, and Rough Fractional Stochastic Volatility models [3, 4, 5, 6]. While these models are theoretically sound, they are computationally demanding to calibrate and generally do not have closed-form solutions, making direct forecasting impracticable.

This motivates a data-driven alternative. SDEs, which driven by inferred unobserved stochastic process, can often also be reformulated as a controlled differential equation (CDE), where the driving signal is instead a fixed observed path. Most importantly, the solution map of these CDEs is entirely determined by the signature of its driving path, motivating the importance of path signatures. The path signature captures all of the geometric and statistical properties of the path meaning that rather than specifying and calibrating a parametric SDE, we can instead learn the mapping from signature features to volatility directly from the data, with no parametric assumptions about the volatility dynamics needed. In Theorem 1 the theoretical justification for this (known as the Universal Approximation Theorem) is explained further in detail.

In this paper, the signature transformed features are used with machine learning techniques to forecast realized volatility. The signature is also useful as it contains properties such as the lead-lag transformation (introduced in Definition 4) which is particularly well suited to this problem as its associated geometric quantity (the Lévy Area 3)is closely related the realized volatility. This gives our approach a direct and interpretable connection to the volatility it aims to forecast, while remaining fully data-driven.

2 Primer on Signatures and Rough Path Theory

In this section we provide the mathematical background required for the remainder of the paper. We begin by introducing the broader context of stochastic and controlled differential equations then define the signatures formally and discuss its key properties.

2.1 From SDEs to Controlled Differential Equations

The volatility models referenced in the introduction (Heston, SABR, rough fractional stochastic volatility) are all formulated as stochastic differential equations (SDEs). An SDE describes how a state process evolves over time under the influence of deterministic drift and stochastic noise, typically driven by a Brownian motion or fractional Brownian motion. While SDEs are powerful modeling tools the stochastic driver is unobserved and must be inferred, and calibrating the model parameters to market data is computationally intensive.

A controlled differential equation (CDE) reformulates this problem with a pathwise representation. Rather than treating the driving signal as a stochastic process, a CDE treats it as a fixed, observed path. Given a driving signal $X_t = (X_t^1, \dots, X_t^d)$, the state process Y_t evolves deterministically according to

$$dY_t = \sum_{i=1}^{d} V_i(Y_t)\, dX_t^i,$$

where $V_1, \dots, V_d$ are vector fields on $\mathbb{R}^e$. Many SDEs have a corresponding CDE formulation in which the stochastic driver is replaced by a realization of the driving path. When the driving path X has finite p-variation with $p < 2$, existence and uniqueness of solutions to the CDE follow from Picard iteration in the Young integration framework. The extension to rougher paths, where $p \geq 2$, requires the machinery of rough path theory and is discussed in the Appendix.[1]

CDEs are related to signatures as the solution map of a CDE is entirely determined by the signature of its driving path. That is, two driving paths that share the same signature will produce identical state trajectories, regardless of their pointwise differences [7]. As a result, rather than solving the

[1]The Appendix covering rough path theory is currently omitted from this preprint and will be included in a later version.

CDE explicitly, one can instead learn the functional mapping from the path signature to forecast target directly from the data. We now introduce the definition of a signature.

2.2 Definition of a Signature

Definition 1 (Signature of a Path)**.** *Let $X_t : [a, b] \to \mathbb{R}^d$ be a path with $t \in [a, b]$. The signature of the path X, denoted by $Sig(X)$, is the vector of iterated integrals:*

$$Sig(X) = \left(1,\ \int_a^b dX_t,\ \int_{a<t_1<t_2<b} dX_{t_1} \otimes dX_{t_2},\ \ldots,\ \int_{a<t_1<\cdots<t_k<b} dX_{t_1} \otimes \cdots \otimes dX_{t_k}\right).$$

where $\otimes$ denotes the tensor product of the differentials. As a result of using the tensor product, our signature becomes a infinite vector of tensors.

Each element of the signature vector corresponds to a level of the signature: the 0^{th} level is simply 1, the first level denoted by $\int_a^b dX_t$ captures the net displacement of the path, and higher levels encodes more complex geometric information about the path. In practice, since the full signature is infinite-dimensional, it cannot be computed directly. Instead we truncate at some finite level k, retaining only the first k levels of iterated integrals. The choice of integral depends on the smoothness of the path. For smooth paths the Riemann integral works, while for non-smooth or rough paths, the Itô, Stratonovich, or Young integral is used depending on the setting.

2.3 Interpretation of Signature

To fully understand the signature, we must introduce some of the key properties including interpretations of levels of the signature as well as key properties. The following section defines levels of the signature as well as covers the interpretation behind the first two levels and corollaries that arise from it.

Definition 2 (Signature Level)**.** *The n-th level of the signature of a path X, denoted by $Sig_{a,b}^{(n)}(X_t)$, is the iterated integral*

$$Sig_{a,b}^{(n)}(X) = \int_{a<t_1<\cdots<t_n<b} dX_{t_1} \otimes \cdots \otimes dX_{t_n}.$$

where $Sig_{a,b}^{(n)}(X) \in (\mathbb{R}^d)^{\otimes n}$.

To better understand the intuition behind the signature, we start with an example where our path is a linear function taking the form $X_t = t$.

2.3.1 Examples

Example 2.1 (One-Dimensional Path): Let $X_t = t$ be a path. At level 0, our signature is 1. At level 1, our signature becomes

$$Sig_{a,b}^{(1)}(X) = \int_a^b dX_t = X_b - X_a$$

Since $X_t = t$, we have $b - a$ which represents the total displacement of the path. The second level of our path is computed as

$$Sig^{(2)}_{a,b}(X) = \int_a^b \int_a^{t_2} dX_{t_1} dX_{t_2} = \frac{(b-a)^2}{2}$$

The second level measures the order in which different parts of the path move. In our one-dimensional case path $X_t = t$, the second level can be thought of as a second order accumulation of increments. In higher dimensions, the second level can help capture area of the path. For this linear path, the one can observe that the nth level of the signature can be written as $\frac{(b-a)^n}{n!}$ which is analogous to the Taylor expansion of e^x. Our following example builds off by introducing the signature levels of a two-dimensional path.

Example 2.2: (Two-Dimensional Circular Path) Consider the path $X_t = (X_t^1, X_t^2)$ where $X_t^1 = cos(t)$ and $X_t^2 = sin(t)$ on the interval $t \in [0, 2\pi]$. Our level one signature of the X_t is computed as

$$Sig^{(1)}_{0,2\pi}(X) = \int_0^{2\pi} dX_t = \left(\int_0^{2\pi} dX_t^1, \int_0^{2\pi} dX_t^2 \right) = \left(\int_0^{2\pi} -\sin(t)\,dt, \int_0^{2\pi} \cos(t)\,dt \right)$$

$$= (\cos(0) - \cos(2\pi),\ \sin(2\pi) - \sin(0)) = (0, 0)$$

On the interval $(0, 2\pi)$, both $cos(t)$ and $sin(t)$ return to their original starting point. Since the first level of the signature measures displacement and both $cos(t)$ and $sin(t)$ return to their original starting point, the total displacement is 0 causing the first level to be $(0, 0)$. The second level can be calculated as

$$\mathrm{Sig}^{(2)}_{0,2\pi}(X) = \begin{pmatrix} \int_0^{2\pi} \int_0^{t_2} dX^1_{t_1} dX^1_{t_2} & \int_0^{2\pi} \int_0^{t_2} dX^1_{t_1} dX^2_{t_2} \\ \int_0^{2\pi} \int_0^{t_2} dX^2_{t_1} dX^1_{t_2} & \int_0^{2\pi} \int_0^{t_2} dX^2_{t_1} dX^2_{t_2} \end{pmatrix}$$

$$= \begin{pmatrix} \int_0^{2\pi} \int_0^{t_2} (-\sin t_1)(-\sin t_2)\,dt_1 dt_2 & \int_0^{2\pi} \int_0^{t_2} (-\sin t_1)(\cos t_2)\,dt_1 dt_2 \\ \int_0^{2\pi} \int_0^{t_2} (\cos t_1)(-\sin t_2)\,dt_1 dt_2 & \int_0^{2\pi} \int_0^{t_2} (\cos t_1)(\cos t_2)\,dt_1 dt_2 \end{pmatrix} = \begin{pmatrix} 0 & \pi \\ -\pi & 0 \end{pmatrix}$$

In example 2.1 the notion that the second level of the signature can encode area was introduced. This area is known as the Lévy Area.

Definition 3 (Lévy Area in $\mathbb{R}^2$)**.** *Let $X : [a, b] \to \mathbb{R}^2$ be a continuous path of bounded variation, with components $X_t = (X_t^1, X_t^2)$. The Lévy area of X over $[a, b]$ is defined by*

$$\mathcal{A}_{a,b}(X) := \frac{1}{2} \left(\int_a^b \int_a^{t_2} dX^1_{t_1}\, dX^2_{t_2} - \int_a^b \int_a^{t_2} dX^2_{t_1}\, dX^1_{t_2} \right).$$

Equivalently,

$$\mathcal{A}_{a,b}(X) = \frac{1}{2} \left(\mathrm{Sig}^{(2)}_{a,b}(X)^{1,2} - \mathrm{Sig}^{(2)}_{a,b}(X)^{2,1} \right),$$

where $Sig^{(2)}_{a,b}(X)^{1,2}$ and $Sig^{(2)}_{a,b}(X)^{2,1}$ represent the top right and the bottom left elements of the second level respectively.

From example 2.2, $X_t = (\cos(t), \sin(t))$ has a Lévy area of π which represents how much area the path sweeps out as well as the direction. On the interval $[0, 2\pi]$, the path $X_t = (cos(t), sin(t))$ traces around a circle with an area π which corresponds exactly to the Lévy area. The use of Lévy Area becomes more apparent when we introduce the lead lag transformation of a path.

Definition 4 (Lead-Lag Transformation of a Path)**.** *Let $X : [a,b] \to \mathbb{R}^d$ be a path observed on a partition $a = t_0 < t_1 < \cdots < t_n = b$. The* lead–lag transformation *of X, denoted by $X^{\mathrm{LL}} : [a,b] \to \mathbb{R}^{2d}$, is the path defined by the sequence*

$$X^{\mathrm{LL}} = \big(X_{t_0}, X_{t_0}\big),\ \big(X_{t_1}, X_{t_0}\big),\ \big(X_{t_1}, X_{t_1}\big),\ \big(X_{t_2}, X_{t_1}\big),\ \big(X_{t_2}, X_{t_2}\big),\ \ldots,\ \big(X_{t_n}, X_{t_n}\big),$$

where each pair $(X_{t_i}, X_{t_j}) \in \mathbb{R}^{2d}$ consists of a lead *component and a* lag *component of the original path. The lead component advances to the next observation while the lag component remains at the previous value, producing a piecewise linear path in $\mathbb{R}^{2d}$.*

In the context of this paper, the lead-lag transformation is valuable as it allows us to capture the quadratic variation of a path (Gyurkó et al., 2013). For a one-dimensional price path, the quadratic variation is proportional to realized volatility which is what we are trying to predict.

2.4 Universal Approximation Theorem

Theorem 1. ***(Universal Approximation Theorem for Signatures).*** *Let $F : C([0,T], \mathbb{R}^d) \to \mathbb{R}$ be a continuous functional, and let $K \subset C([0,T], \mathbb{R}^d)$ be compact. Then for every $\epsilon > 0$, there exists a linear functional L on the tensor algebra such that for all $a \in K$,*

$$|F(a) - L(\mathrm{Sig}(a))| < \epsilon.$$

A proof of this result can be found in Levin, Lyons, and Ni [1], where the signature approximation can be proved using the shuffle product identity and the Stone–Weierstrass theorem. The Universal Approximation Theorem for signatures establishes that linear functionals of the signatures are dense in the space of continuous functionals on paths. As a result, any path dependent quantity can be approximated arbitrarily well using a linear model on these signature features.

2.5 Signature Kernel of Paths

Definition 5 (Signature Kernel of Two Paths). Let $X : [a,b] \to \mathbb{R}^d$ and $Y : [a,b] \to \mathbb{R}^d$ be two paths on an interval $[a,b]$. The signature kernel of X and Y is defined as:

$$K(\mathrm{Sig}(X), \mathrm{Sig}(Y)) = \sum_{k=0}^{\infty} \langle \mathrm{Sig}^k(X), \mathrm{Sig}^k(Y) \rangle. \tag{1}$$

In practice, since signatures are infinite-dimensional, we can either truncate at level N such that the kernel becomes:

$$K_N(\mathrm{Sig}(X), \mathrm{Sig}(Y)) = \sum_{k=0}^{N} \langle \mathrm{Sig}^k(X), \mathrm{Sig}^k(Y) \rangle. \tag{2}$$

Alternatively, we can introduce a discount parameter $\gamma \in (0,1)$ resulting in our kernel being:

$$K_\gamma(\mathrm{Sig}(X), \mathrm{Sig}(Y)) = \sum_{k=0}^{\infty} \gamma^k \langle \mathrm{Sig}^k(X), \mathrm{Sig}^k(Y) \rangle. \tag{3}$$

In (9), our signatures are truncated to become finite dimensional, allowing the kernel to be computable. In (3), our discount factor becomes a penalty for higher levels of our signature. Since $\gamma \in (0,1)$, as k approaches infinity, our discount factor γ^k approaches 0 down-weighing higher levels of the signature.

3 Existing Methods

3.1 Latent Volatility Models

The first class of volatility models were latent volatility models which tried to infer volatility from market returns. In 1982, Robert Engle formalized the first model known as the Autoregressive Conditional Heteroskedasticity (ARCH) model [2]. This model assumed that market returns could be modeled as $r_t = \mu + \epsilon_t$ where $\epsilon_t = \sigma_t z_t$ and $z_t \sim \mathcal{N}(0,1)$. Under the ARCH model, at a time t, the latent volatility is modeled as

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \cdots + \alpha_q \epsilon_{t-q}^2 \tag{4}$$

for some parameter q representing how many past squared residuals are used and weights $\alpha_1, \ldots, \alpha_q > 0$ representing the relative contribution of each lagged squared residual to the current conditional variance. Shortly after, the GARCH model [17] built off ARCH by adding lagged variance terms, capturing volatility clustering more efficiently. Based on the GARCH model, at time t, an asset's volatility was modeled as

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2 \tag{5}$$

where weights $\alpha_1, \ldots, \alpha_q > 0$ have the same interpretation as in 4 and $\beta_1, \ldots, \beta_p > 0$ represent the contribution of the past conditional variances σ_{t-j}^2. In both models, the p and q parameters are chosen by the user.

3.2 Realized Volatility Models

These models were later followed by realized volatility (RV) approaches, which construct volatility directly from intraday returns, avoiding the need to infer an unobserved latent process. By doing so, the reliance on restrictive parametric assumptions in the ARCH and GARCH models was removed.

The main common RV models used today are the Heterogeneous Auto-Regressive (HAR) class of models proposed by Corsi in 2009 [18]. The most simple case of the HAR model is the Heterogeneous Auto-Regressive Realized Volatility (HAR-RV) model where realized volatility is modeled as

$$RV_{t+1} = \beta_0 + \beta_d RV_t^{(d)} + \beta_w RV_t^{(w)} + \beta_m RV_t^{(m)} + \epsilon_{t+1}$$

In this model, $RV_t^{(d)}, RV_t^{(w)}, RV_t^{(m)}$ represent the daily, weekly, and monthly realized volatilities and $\beta_0, \beta_d, \beta_w, \beta_m$ are the corresponding parameters that are usually fit with Ordinary Least Squares (OLS). Generally, d, w, t take on the values $1, 5, 22$ respectively as there are 1, 5, and 22 trading days in the last day, week, and month. Though extensions of the HAR models [19, 20] have been proposed to deal with volatility with jumps or with long memory, the standard HAR-RV provides a general baseline in our case.

The GARCH model was also extended in a RV sense through the introduction of GARCH-RV models, which incorporate realized volatility measurements into the conditional variance dynamics. A standard GARCH-RV(1,1) model takes the form

$$RV_{t+1} = \sigma_{t+1}^2 + \epsilon_{t+1},$$

where the conditional variance evolves according to

$$\sigma_{t+1}^2 = \omega + \alpha\epsilon_t^2 + \beta\sigma_t^2.$$

Here, $\omega > 0$ is a constant term, $\alpha > 0$ measures the contribution of previous volatility shocks, and $\beta > 0$ captures the persistence of past conditional variances. The parameters are typically estimated using quasi-maximum likelihood estimation (QMLE) since returns generally have fat tails and are not Gaussian. Similar to the standard GARCH framework, GARCH-RV models are designed to capture volatility clustering commonly observed in financial time series.

3.3 Machine Learning Methods

Machine learning methods branch off from realized volatility models by incorporating additional features such as trading volume [21, 22], macroeconomic indicators [21], order flow information, option-implied volatility, and other high-dimensional market features. These methods aim to capture nonlinear relationships and complex interactions that may not be adequately modeled by traditional linear econometric approaches such as HAR-RV or GARCH models.

Typically, the machine learning models are grouped into the following categories: tree-based methods (random forests, gradient boosted trees), kernel methods (support vector regression), neural networks (LSTM), and hybrid models that combine economic structure with learned components. In such methods, feature engineering places a large role in how well the models actually perform with raw data being insufficient. This leads to additional engineering challenges such as having to construct a sufficient feature map as well as finding the data for those corresponding features. In addition, these models may be overfit to data, have poor interpretability, or not be robust to data.

4 Signature Based Volatility Models

4.1 Feature Construction

To construct signature features, we first specify a lookback window which determines the number of past bars used to form the return path. This return path is then embedded into $\mathbb{R}^2$ via the lead–lag transformation from Definition 4, and the signature is computed on this resulting lead–lag path. Rather than using a single fixed window, we consider three lookback windows $\ell_1 < \ell_2 < \ell_3$ representing short, medium, and long-range historical information respectively. We then set $\ell^* =$

$\max(\ell_1, \ell_2, \ell_3) = \ell_3$ as the longest lookback window, which determines the path length used in feature construction.

For each time t, we compute the signature at depth N of the lead-lag path over a single lookback window of ℓ^* bars:

$$\mathbf{x}_t = \mathrm{Sig}^N\big(X^{\mathrm{LL}}_{t-\ell^*+1:t}\big) \;\in\; \mathbb{R}^{M(2,N)}, \tag{6}$$

where $X^{\mathrm{LL}}_{t-\ell^*+1:t}$ denotes the lead-lag transformation of the return path and $M(2, N)$ denotes the dimension of the truncated signature of a two-dimensional path at depth N. Signatures are computed using the `iisignature` library [8], which implements the piecewise-linear Stratonovich signature exactly. The forecast target is the h-bar-ahead realized volatility:

$$\mathrm{RV}_t^{(h)} = \sqrt{\sum_{i=1}^{h} r_{t+i}^2}, \quad h = 5. \tag{7}$$

4.2 Two Stage Lasso–Signature Kernel Regression

Since the signature feature space can be very high-dimensional depending on the level of signature we truncate at, we follow the two-step approach of Guo et al. [16] to make the regression tractable.

Stage 1: Lasso feature selection. All feature dimensions and the target are standardized. Lasso cross-validates the regularization strength α^* over 100 values using an inner time-series cross-validation with five folds:

$$\hat{\beta} = \arg\min_{\beta} \left\{ \frac{1}{n}\|y - X\beta\|_2^2 + \alpha^* \|\beta\|_1 \right\}. \tag{8}$$

By using Lasso, we can perform feature selection on our features while also fitting an optimal set of $\hat{\beta}$ parameters. Any feature dimension with $|\hat{\beta}_j| > 10^{-10}$ is kept while the others are discarded. This allows us to keep only the signature features that are predictive of the target.

Stage 2: Signature kernel regression. Let $\mathcal{I} = \{j : |\hat{\beta}_j| > 10^{-10}\}$ be the selected index set. On the retained signature coordinates $\mathbf{x}_t^{\mathcal{I}}$, we fit Kernel Ridge Regression with the truncated signature kernel:

$$K_N(\mathrm{Sig}(X), \mathrm{Sig}(Y)) = \sum_{k=0}^{N} \langle \mathrm{Sig}^k(X),\, \mathrm{Sig}^k(Y)\rangle. \tag{9}$$

Since the retained feature vectors $\mathbf{x}^{\mathcal{I}}$ are explicit coordinates in the truncated tensor algebra, their dot product is the signature kernel by definition. The KRR solution minimizes

$$\min_{\mathbf{w}} \sum_{i=1}^{n} \big(y_i - \mathbf{w}^\top K_N(\mathbf{x}_i, \cdot)\big)^2 + \lambda \|\mathbf{w}\|^2, \tag{10}$$

where $\lambda \in \{10^{-4}, 10^{-2}, 1, 10, 100\}$ is selected by inner time-series grid search. The KRR prediction takes the dual form

$$\hat{y}(\mathbf{x}) = \sum_{i=1}^{n} w_i\, K_N(\mathbf{x}, \mathbf{x}_i), \tag{11}$$

weighting each training path $\mathbf{x}_i$ by its signature-kernel similarity to the query path $\mathbf{x}$. If Lasso eliminates all features, then our Stage 1 predictions are used as a fallback.

4.3 Walk-Forward Adaptive Estimation

A key challenge is that in volatility forecasting, the return process is often non-stationary. Different regimes, clustering intensity, and feature significance can change over time making past parameters obsolete. Guo et al. [16] addresses this through adaptive weighting via the signature kernel distance, increasing weights for training samples whose path signatures are similar to the current period. We implement a similar adaptation through a rolling-window walk-forward procedure.

Let W denote the window size and Δ denote the number of new observations collected between successive retraining steps. Starting from an initial training set of $T_0 = 200$ observations, the full Lasso–KRR pipeline is refit every Δ new observations using only the most recent W training samples:

$$\text{train on } \{(\mathbf{x}_s, y_s)\}_{s=t-W}^{t-1}, \quad \text{predict } \{y_s\}_{s=t}^{t+\Delta-1}, \quad t \leftarrow t + \Delta. \tag{12}$$

We set $\Delta = \ell^*$ (one full long-window cycle) and $W = 5\ell^*$ (five cycles), so the model retains approximately five repetitions of the dominant timescale. This bounds the effective memory of the signature kernel sum, preventing the dual weights w_i from being influenced by path observations in stale volatility regimes — the rolling window plays the role of the exponential decay in the signature kernel distance weights of [16].

5 Data and Methodology

5.1 Data

In our numerical experiments, we used open-high-low-close-volume (OHLCV) bar data from yfinance [23] at five sampling frequencies: 1 min, 5 min, 10 min, 1 hour, and 1 day. The available history of each frequency varies from 7 calendar days at 1 minute, 60 calendar dates at 5 and 10 minutes, 730 days at 1 hour, and 10 years at the daily. Since yfinance doesn't provide 10 minute bars, we construct them from the 5 minute bars by taking the open at the first sub-bar, the high and the low as the maximum and minimum of the sub-bars, and the close of the last sub-bar.

5.2 Lead–Lag Transformation of the Return Path

Let $r_t = \ln(C_t/C_{t-1})$ denote the log-return at bar t, where C_t is the closing price at t. At each time t, we form the lookback window $\mathbf{r} = (r_{t-\ell^*+1}, \ldots, r_t) \in \mathbb{R}^{\ell^*}$ from the price series and apply the lead–lag transformation from Definition 4 to embed this one-dimensional return path into $\mathbb{R}^2$. The resulting path $\mathbf{r}^{\mathrm{LL}} \in \mathbb{R}^{(2\ell^*-1)\times 2}$ traces L-shaped steps in the (X^1, X^2) plane, where X^1 (lead) and X^2 (lag) are both in units of log-returns.

The motivation for this embedding is twofold. First, as mentioned in Definition 4, the Lévy area of the lead–lag path equals half the quadratic variation of the original return path, which is directly proportional to our target variable which is realized volatility. Second, the signature of the lead–lag transformation also encodes the *directional* decomposition of quadratic variation: increments where $X^1 > X^2$ (above the diagonal) correspond to upward price movement, and increments where $X^1 < X^2$ (below the diagonal) correspond to downward movement. The signed difference between these two areas captures momentum and mean-reversion structure that is unavailable to the realized variance measures used in HAR-type models.

5.3 Path Signature Features

For the lead–lag transformed path $X^{\mathrm{LL}} = (X^{\mathrm{LL},1}, X^{\mathrm{LL},2})$ with increments $dX_s^{\mathrm{LL}} = (dX_s^{\mathrm{LL},1}, dX_s^{\mathrm{LL},2})$, we define the signature levels as follows.

The level 1 signature is:

$$\mathrm{Sig}_{a,b}^{(1)}(X^{\mathrm{LL}}) = \left(\int_a^b dX_s^{\mathrm{LL},1},\ \int_a^b dX_s^{\mathrm{LL},2} \right).$$

The level 2 signature is:

$$\mathrm{Sig}_{a,b}^{(2)}(X^{\mathrm{LL}}) = \left(\int_{a<s<u<b} dX_s^{\mathrm{LL},i} \otimes dX_u^{\mathrm{LL},j} \right)_{i,j\in\{1,2\}}.$$

The level 3 signature is:

$$\mathrm{Sig}_{a,b}^{(3)}(X^{\mathrm{LL}}) = \left(\int_{a<s_1<s_2<s_3<b} dX_{s_1}^{\mathrm{LL},i} \otimes dX_{s_2}^{\mathrm{LL},j} \otimes dX_{s_3}^{\mathrm{LL},k} \right)_{i,j,k\in\{1,2\}}.$$

In our numerical experiments in Section 6, we truncate the signature at level $m = 3$. The truncated signature of the lead–lag transformed path X^{LL} then takes values in the truncated tensor algebra

$$T^{(m)}(\mathbb{R}^{2d}) = \bigoplus_{k=0}^{m} (\mathbb{R}^{2d})^{\otimes k},$$

which at $m = 3$ yields a finite-dimensional feature representation consisting of level 1, level 2, and level 3 iterated integrals.

5.4 Benchmark Models

We compare against the *HAR-RV* model of [18], which regresses future realized volatility on three rolling volatility measures at daily, weekly, and monthly scales:

$$RV_{t+1} = \beta_0 + \beta_d\, RV_t^{(d)} + \beta_w\, RV_t^{(w)} + \beta_m\, RV_t^{(m)} + \varepsilon_{t+1}, \tag{13}$$

where

$$RV_t^{(\ell)} = \sqrt{\frac{1}{\ell} \sum_{i=0}^{\ell-1} r_{t-i}^2},$$

and $\ell \in \{1, 5, 22\}$ corresponds to the daily, weekly, and monthly realized volatility windows respectively. As mentioned in Section 3.2, $\beta_0, \beta_d, \beta_w, \beta_m$ are fit on the features using OLS.

We additionally include a *GARCH-X(1,1)* benchmark, in which the conditional variance follows as

$$h_t = \omega + \alpha\, \mathrm{rv}_{t-1} + \beta\, h_{t-1}, \tag{14}$$

with parameters estimated by quasi-maximum likelihood and $\mathrm{rv}_t = r_t^2$ being the one-step realized variance proxy. Unlike HAR-RV, which directly targets realized volatility, GARCH-X(1,1) models the latent conditional variance h_t and uses it as a one-step-ahead volatility forecast. Both benchmarks ultimately produce a scalar volatility forecast that is evaluated against $\mathrm{RV}_t^{(h)}$ using the same out-of-sample metrics.

Finally, we include an *XGBoost* benchmark that uses the same realized volatility feature set as HAR-RV $\left(RV_t^d,\, RV_t^w,\, RV_t^m\right)$ but replaces the linear OLS estimator with gradient-boosted regression trees [25]. By doing so, we capture the nonlinearity structure of volatility that the traditional HAR-RV model might have missed. Since *XGBoost* uses the same feature set as HAR-RV, any difference between the two is attributable to the model class rather than feature representation. The trees are fitted with depth four, learning rate 0.05, and column subsampling rate 0.8 to limit overfitting on the relatively small realized-volatility samples available at each frequency. All three benchmarks are evaluated under the same `TimeSeriesSplit` out-of-sample protocol as the signature model.

5.5 Evaluation Protocol

All models are evaluated on out of sample performance. To ensure that models are being trained on the same data, predictions from all models are aligned on their common set of timestamps with an inner join before metrics are computed. We report mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and the QLIKE loss defined as

$$\mathrm{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left[\ln\hat{\sigma}_t + \frac{\sigma_t}{\hat{\sigma}_t}\right], \tag{15}$$

which is standard in the realized volatility literature [24]. The QLIKE metric penalizes underprediction of volatility more severely than overprediction, reflecting the reality of underestimating risk. Lower values of QLIKE score indicate a better model for volatilty forecasting.

6 Numerical Results

6.1 Dataset and Experimental Setup

We evaluate our models on the S&P 500 ETF (`SPY`) across five sampling frequencies: 1 min, 5 min, 10 min, 1 hour, and 1 day. Table 6.1 summarizes the realized volatility profile at each frequency. Since our intraday history is limited by the Yahoo Finance API (7 days at 1-minute, 60 days at 5- and 10-minute, 730 days at 1-hour), the following results are more indicative rather than definitive.

Frequency	Obs.	Mean RV (%)	Std RV (%)	Median RV (%)	Ann. Vol. (%)
1-minute	2,607	0.0503	0.0483	0.0399	7.05
5-minute	4,655	0.1786	0.1775	0.1347	11.20
10-minute	2,327	0.2661	0.2460	0.2026	11.80
1-hour	5,075	0.6160	0.5075	0.4799	11.15
1-day	2,513	1.9831	1.6047	1.6219	14.08

Table 1: Realized-volatility profile for SPY by sampling frequency.

All models are evaluated using a TimeSeriesSplit with $K = 5$ folds, ensuring strictly out-of-sample (OOS) predictions with no look-ahead bias. Predictions from all models are aligned with an inner join on timestamps before metrics are computed, allowing every model in Table 2 to be evaluated the same n observations at each frequency. The forecast target is the 5-bar-ahead realized volatility defined in Equation 7.

We report four metrics: mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and the QLIKE score (as defined in 15), which is the standard loss in the volatility-forecasting literature as it penalizes directional misprediction asymmetrically [24].

6.2 Out-of-Sample Results

Freq	Model	n	Lookback	RMSE (%)	MAE (%)	QLIKE
1m	`SIG`	2250	195	0.04855	0.02483	−2.038
	`HAR_RV`	2250	—	**0.04726**	**0.02246**	**−2.077**
	`XGBOOST`	2250	—	0.05151	0.02413	−2.062
	`GARCH`	2250	—	0.06647	0.03913	−2.015
5m	`SIG`	3875	78	**0.1824**	**0.0981**	**−0.754**
	`HAR_RV`	3875	—	0.1864	0.1038	−0.732
	`XGBOOST`	3875	—	0.1868	0.1016	−0.728
	`GARCH`	3875	—	0.2441	0.1987	−0.581
10m	`SIG`	1925	39	**0.2514**	**0.1511**	**−0.352**
	`HAR_RV`	1925	—	0.2622	0.1696	−0.266
	`XGBOOST`	1925	—	0.2740	0.1774	−0.286
	`GARCH`	1925	—	0.3345	0.2848	−0.185
1h	`SIG`	4205	35	0.5221	0.3458	+0.518
	`HAR_RV`	4205	—	**0.4802**	**0.3118**	**+0.486**
	`XGBOOST`	4205	—	0.5116	0.3317	+0.517
	`GARCH`	4205	—	0.5580	0.4592	+0.554
1d	`SIG`	2070	60	1.769	1.112	+1.812
	`HAR_RV`	2070	—	**1.285**	**0.777**	**+1.710**
	`XGBOOST`	2070	—	1.600	0.883	+1.755
	`GARCH`	2070	—	1.446	1.002	+1.711

Table 2: Out-of-sample forecast metrics for SPY at five sampling frequencies. RMSE and MAE are reported in percentage units (%). Bold denotes the best value in each column.

Figure 1 plots these results for RMSE, MAE, R^2, and QLIKE across all five frequencies for each model.

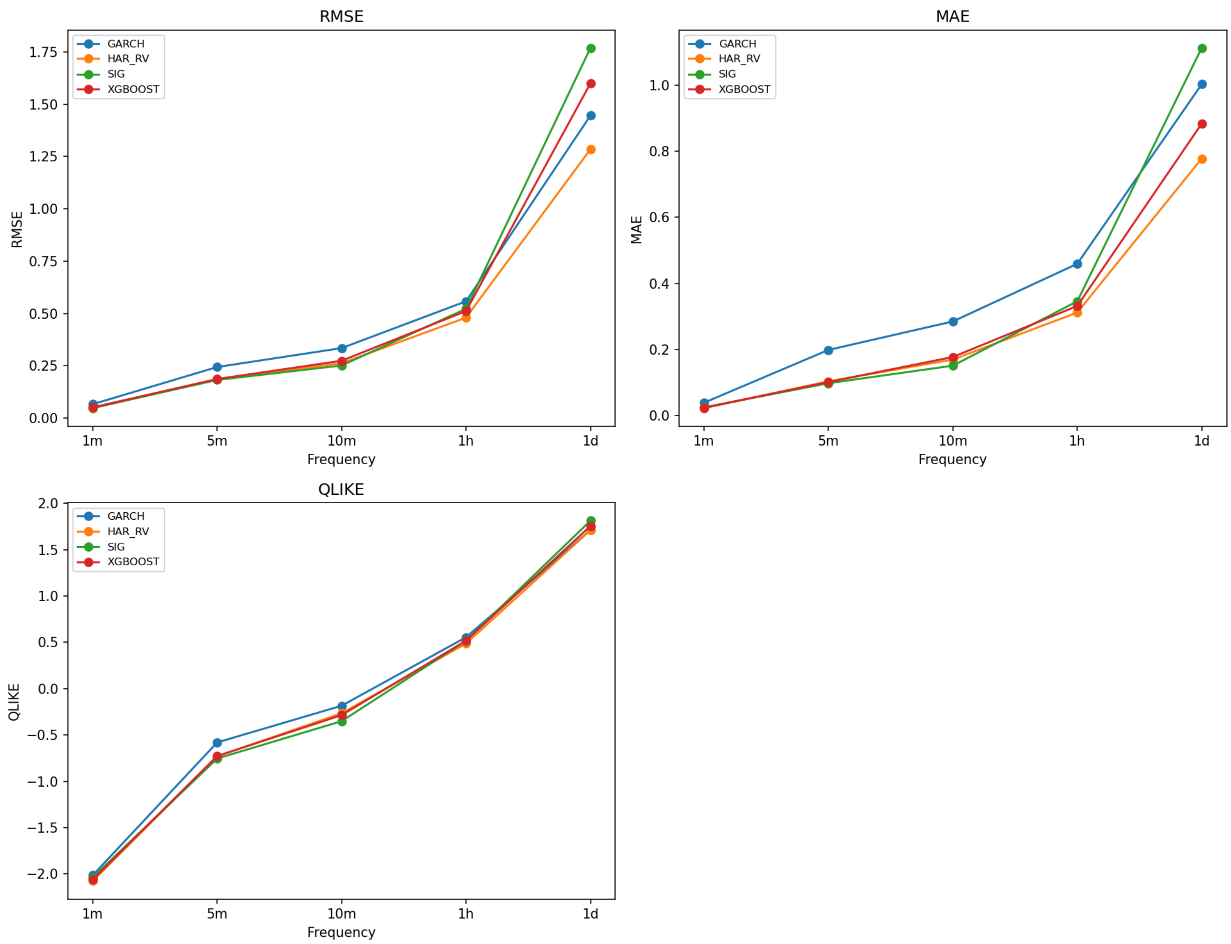

Figure 1: Out-of-sample forecast metrics for SPY across all five sampling frequencies.

6.3 Discussion

This subsection analyzes our results. We first analyze the RMSE and MAE together and then look at the QLIKE scores across models and frequencies.

6.3.1 RMSE and MAE

Daily frequency. At the daily horizon, HAR-RV achieves the lowest RMSE (1.285%) and MAE (0.777%), consistent with the well-established finding that daily realized volatility is highly autocorrelated at weekly and monthly lags [11]. GARCH provides a competitive secondary benchmark (RMSE = 1.446%, MAE = 1.002%), confirming that the conditional variance recursion captures meaningful long-run volatility clustering. SIG records the highest RMSE (1.769%), falling well short of all three benchmarks. We attribute this to the level-3 truncation of the path signature: at $N = 3$, the feature map produces fourteen coordinates per lookback window (two level-1 integrals, four level-2 iterated integrals, and eight level-3 iterated integrals), which captures displacement, path area, and third-order geometric structure of the return path, but remains insufficient to represent the multi-week autocorrelation structure that HAR-RV encodes directly through its rolling-window features. XGBoost achieves RMSE = 1.600%, sitting between HAR-RV and GARCH, suggesting that nonlinearity offers modest but limited gains over the linear HAR feature set at this timescale.

Hourly frequency. At the 1-hour frequency, HAR-RV remains the strongest model (RMSE = 0.480%, MAE = 0.312%). SIG outperforms GARCH on both RMSE (0.522% versus 0.558%) and MAE (0.346% versus 0.459%), indicating that the rolling-window refit mechanism adapts better to intraday regime changes than the GARCH variance recursion. XGBoost ranks between HAR-RV and SIG by RMSE (0.512%), again underscoring that nonlinearity does not materially improve on the linear HAR feature set at this timescale.

Intraday frequencies (1-minute, 5-minute, 10-minute). At intraday scales the picture shifts in favour of SIG. At 1-minute, HAR-RV records the lowest RMSE (0.047%) and MAE (0.022%), with SIG close behind (0.049% and 0.025%). XGBoost underperforms at this frequency, suggesting the gradient-boosted trees overfit the small intraday sample. At 5-minute, SIG achieves the best RMSE (0.182%) and MAE (0.098%) across all models, with HAR-RV and XGBoost close behind at 0.186% and 0.187% respectively. At 10-minute, SIG again leads on both RMSE (0.251%) and MAE (0.151%), while GARCH deteriorates sharply (RMSE = 0.334%, MAE = 0.285%), confirming that the GARCH(1,1) variance recursion is not suited to microstructure-dominated regimes.

6.3.2 QLIKE

Daily and hourly frequencies. At the daily horizon, HAR-RV achieves the lowest QLIKE (+1.710), with GARCH close behind (+1.711) — a negligible difference suggesting both models manage underprediction risk similarly at this scale. SIG records the highest QLIKE (+1.812), consistent with its poor RMSE performance and indicating that it disproportionately underpredicts volatility at the daily horizon. XGBoost sits in between at +1.755. At the 1-hour frequency, HAR-RV again leads (+0.486), with SIG (+0.518) and XGBoost (+0.517) nearly identical, and GARCH the weakest (+0.554).

Intraday frequencies (1-minute, 5-minute, 10-minute). At intraday scales the QLIKE ranking aligns closely with RMSE. At 1-minute, HAR-RV achieves the best QLIKE (−2.077), with SIG second (−2.038), XGBoost third (−2.062), and GARCH weakest (−2.015). At 5-minute and 10-minute, SIG leads on QLIKE (−0.754 and −0.352 respectively), with HAR-RV and XGBoost close behind. GARCH records the weakest QLIKE across all three intraday horizons (−0.581 and −0.185), confirming its poor calibration at high frequencies.

Signature truncation level. A limitation of the present results is that all signature computations use truncation level $N = 3$, yielding fourteen features per lookback window. At this level the signature captures total displacement, path area (the Lévy area), and third-order path geometry, but discards higher-order iterated integrals that encode finer curvature and oscillatory structure of the return path. Increasing to $N = 4$ or $N = 5$ expands the feature dimension to 30 and 62 coordinates respectively for a two-dimensional lead-lag path, providing substantially richer path representations and, we expect, materially improving the out-of-sample performance of SIG relative to the HAR-RV benchmark, particularly at the daily frequency where multi-scale autocorrelation structure dominates.

7 Conclusion

On intraday forecasts, our numerical results show that the proposed signature method is competitive with existing benchmarks such as the HAR-RV model, winning in the 5 and 10 minute time horizons and being competitive at the 1 minute time horizon. However, in the 1 hour and 1 day time horizon, the proposed signature method falls apart.

One interpretation of these results is that signatures are competitive in intraday forecasting because short term volatility dynamics more geometric in nature, driven mostly by factors such as directional asymmetry, oscillatory behavior, and quadratic variation encoded through the lead lag transformation.

On longer time horizons, however, macroeconomic features play a much larger role in volatility dynamics. Factors such as interest rate hikes, market sentiment, and geopolitical events are not encoded into the price path. This suggests that the underperformance of the signature method is not due to limitations of the signature approach, but due to the result of using a minimal feature set. By expanding our driving path to include the paths of macroeconomic covariates or increasing the truncation level beyond three may close the gap or even improve up on it.

Nonetheless, our numerical experiments on SPY data show that even the most minimal signature approach using only the return path is competitive with HAR-RV and other machine learning methods at at intraday frequencies. This is important as it establishes signatures as a promising path to forecast volatility. Forecasters may build off of this method and result to add additional features such as trading volume or macroeconomic covariates, allowing the signature based methods to consistently outperform HAR-style benchmarks across all frequencies while retaining the interpretability that deep learning models lack.

8 Further Work

There is still lots of work remaining in this project. First, the numerical experiments were limited by the data available through the yfinance API, which only provides short intraday histories. Expanding to a longer and more comprehensive dataset covering multiple tickers and distinct volatility regimes would give a clearer picture of how the signature method generalizes out of sample.

Second, the results presented here lack formal statistical testing. Diebold–Mariano tests [9] should be applied to determine whether the performance differences between SIG and the benchmarks are statistically significant, rather than attributable to sampling variation.

Third, the numerical experiments used a minimal driving path of log-return series embedded with a lead–lag transformation. Both Guo et al. [16] and Christensen et al [21] show that enriching the input path with additional covariates such as trading volume, option-implied volatility, or macroeconomic indicators can substantially improve forecast quality. Incorporating such features into the path is a straightforward extension that may close the performance gap at longer horizons.

Finally, the machine learning component of the pipeline could be strengthened. This paper uses Lasso and kernel ridge regression, but the signature feature representation is compatible with many other methods as well. Further work with deep learning approaches such as signature-based RNNs or neural CDEs may better capture the nonlinear volatility dynamics that the current pipeline might have missed.

References

[1] David Levin, Terry Lyons, and Hao Ni. *Learning from the past, predicting the statistics for the future, learning an evolving system.* arXiv preprint arXiv:1309.0260, 2013. `https://arxiv.org/abs/1309.0260`

[2] R. F. Engle. Autoregressive Conditional Heteroskedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, 50(4):987–1007, 1982. `https://www.jstor.org/stable/1912773`

[3] Heston, S. L. (1993). A closed-form solution for options with stochastic volatility with applications to bond and currency options. *The Review of Financial Studies*, 6(2):327–343.

[4] Abi Jaber, E., Larsson, M., and Pulido, S. (2019). Affine Volterra processes. *The Annals of Applied Probability*, 29(5):3155–3200.

[5] Hagan, P. S., Kumar, D., Lesniewski, A. S., and Woodward, D. E. (2002). Managing smile risk. *Wilmott Magazine*, 1:84–108.

[6] Gatheral, J., Jaisson, T., and Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance*, 18(6):933–949.

[7] Lyons, T. J. (1998). Differential equations driven by rough signals. *Revista Matemática Iberoamericana*, 14(2):215–310.

[8] Jeremy Reizenstein and Benjamin Graham. *Algorithm 1004: The iisignature Library: Efficient Calculation of Iterated-Integral Signatures and Log Signatures.* ACM Transactions on Mathematical Software (TOMS), 46(1), 2020.

[9] Diebold, F. X., and Mariano, R. S. (1995). "Comparing Predictive Accuracy." *Journal of Business & Economic Statistics*, 13(3), 253–263. Stable URL: `https://www.jstor.org/stable/1392185`

[10] F. J. Király and H. Oberhauser. Kernels for Sequentially Ordered Data. *Journal of Machine Learning Research*, 20(31):1–45, 2019.

[11] Andersen, T. G., Bollerslev, T., Diebold, F. X., and Labys, P. (2003). "Modeling and Forecasting Realized Volatility." *Econometrica*, 71(2), 579–625. https://doi.org/10.1111/1468-0262.00418

[12] C. Salvi, T. Cass, J. Foster, and T. Lyons. The Signature Kernel Is the Solution of a Goursat PDE. *SIAM Journal on Mathematics of Data Science*, 3(3):873–899, 2021.

[13] B. Fütter, T. Lyons, and H. Oberhauser. Kernel Learning for Mean-Variance Trading Strategies. *arXiv preprint arXiv:2401.04857*, 2024.

[14] T. Lyons, M. Caruana, and T. Lévy. *Differential Equations Driven by Rough Paths*. Springer, 2007.

[15] I. Chevyrev and A. Kormilitzin. A Primer on the Signature Method in Machine Learning. *arXiv preprint arXiv:1603.03788*, 2016.

[16] H. Gu, X. Guo, T. L. Jacobs, P. Kaminsky, and X. Li. *Transportation Marketplace Rate Forecast Using Signature Transform*. arXiv preprint arXiv:2401.04857, 2024. https://arxiv.org/abs/2401.04857

[17] T. Bollerslev. Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986. https://public.econ.duke.edu/~boller/Published_Papers/joe_86.pdf

[18] F. Corsi. A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2):174–196, 2009. https://doi.org/10.1093/jjfinec/nbp001

[19] Torben G. Andersen, Tim Bollerslev, Francis X. Diebold, and Paul Labys, "Modeling and Forecasting Realized Volatility," NBER Working Paper No. 8160, 2001. Available at SSRN: https://ssrn.com/abstract=847047.

[20] Z. Huang, H. Liu, and T. Wang, "Modeling long memory volatility using realized measures of volatility: A realized HAR GARCH model," *Economic Modelling*, vol. 52, pp. 812–821, 2016. doi: 10.1016/j.econmod.2015.10.018.

[21] K. Christensen, M. Siggaard, and B. Veliyev. A machine learning approach to volatility forecasting. *arXiv preprint*, arXiv:2601.13014, 2026. Available at: https://arxiv.org/abs/2601.13014

[22] Díaz, J., Hansen, E., and Cabrera, G. (2024). Machine learning for stock market volatility: Predictability, drivers, and economic value. *Research in International Business and Finance*, 67, 102115. https://doi.org/10.1016/j.ribaf.2023.102115

[23] Ran Aroussi. *yfinance: Yahoo! Finance Market Data Downloader*. GitHub repository. Available at: https://github.com/ranaroussi/yfinance. Accessed: May 7, 2026.

[24] Patton, A. J. (2011). "Volatility Forecast Comparison Using Imperfect Volatility Proxies." *Journal of Econometrics*, 160(1), 246–256. doi:10.1016/j.jeconom.2010.03.034.

[25] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.